FixProof: A Validation-Oriented System for AI-Generated Vulnerability Remediation

Tony Tran

CS6727 Capstone Project

Partial Final-Report Draft for TA Review - October 27, 2026

Draft status: This is substantive draft work, not the final December submission. The argument, methods, evidence tables, limitations, conclusion, disclosure, and references are present. Final editing, selected figures, page layout, citation-format review, and release-manifest verification remain.

Abstract

AI coding systems can produce plausible vulnerability repairs, but a patch may remove a static-analysis warning while preserving exploitability, or block a tested attack while changing intended behavior. FixProof is a bounded Python orchestration prototype that preserves static-analysis, runtime-security, functional, deterministic-policy, and human-review evidence as separate artifacts. The evaluation uses one controlled JavaScript/Express fixture for each of three weakness classes: reflected cross-site scripting (CWE-79), SQL injection (CWE-89), and path traversal (CWE-22). Five scheduled repair calls were made per fixture, producing 15 primary candidate instances. All 15 passed their complete registered primary security and functional suites, while only five removed the target Semgrep finding. Supplemental testing exposed five behavioral-parity failures, ten robustness-contract failures, and five Windows-inconclusive symbolic-link observations. A separately registered WSL2 extension then showed that all five saved path-traversal candidates disclosed a controlled outside-file marker through a symbolic link. These results do not establish a production repair-success rate. They show, within the registered fixtures, why scanner disappearance, fixed tests, policy routing, and human review should remain distinguishable and auditable.

1. Problem and Research Question

AI-generated security repairs require more evidence than syntactic validity or the disappearance of a scanner warning. A proposed repair can still be exploitable, can reject legitimate inputs, or can satisfy a narrow test while failing at a boundary that was not represented. Allowing the same model that generated a patch to approve that patch also creates an authority problem: plausibility is not independent validation.

FixProof asks the following bounded research question:

Does disappearance of the original static-analysis finding agree with targeted runtime-security and functional evidence for AI-generated repairs in three controlled Express fixtures?

The project does not ask whether an AI model can autonomously secure arbitrary repositories. It studies how multiple evidence channels can be recorded and interpreted without treating any one channel as proof of application-wide security.

2. Related Work and Design Motivation

Pearce et al. (2022) evaluated the security of code generated with GitHub Copilot in selected scenarios and found that plausible generated code can contain security weaknesses. Pearce et al. (2023) studied zero-shot vulnerability repair with large language models. Kulsum et al. (2024) evaluated reasoning and patch-validation feedback in automated vulnerability repair. These works motivate FixProof's focus on evidence after generation, rather than treating generation itself as the final result.

FixProof differs in scope from a claim of a new state-of-the-art repair model. Its contribution is a small, inspectable evidence workflow for coursework: a candidate is generated under fixed conditions; syntax and SAST are recorded; separate runtime-security and benign-behavior validators are executed; a deterministic policy routes the result; and a person records a bounded decision with a rationale. OWASP's Web Security Testing Guide informed the organization of targeted runtime checks, while Semgrep documentation informed scanner execution. Neither is treated as an experimental dataset.

3. Prototype Architecture and Authority Boundary

FixProof is implemented as Python orchestration around three controlled JavaScript/Express applications. For each scheduled primary attempt, the orchestrator records the baseline finding and focused source context, obtains a candidate response, applies it in a copied workspace, and invokes the defined validators. Evidence is stored as JSON rather than collapsed into one score.

The evidence path is:

1. establish the vulnerable baseline and registered attack/control behavior;

2. provide one normalized finding and focused code context to the repair model;
3. apply the candidate in a copied workspace;
4. run syntax, Semgrep, targeted runtime-security, and functional checks;
5. route the recorded result using deterministic policy; and
6. store any human verdict and rationale separately.

The repair model does not approve or deploy its own patch. Python coordinates the stages but is not an independent security judge. Validators measure registered conditions, policy routes those measurements, and the human reviewer accepts, rejects, or requests more evidence within the stated scope. Copied workspaces protect frozen inputs from modification; they are not security sandboxes for arbitrary hostile code.

The public website at https://fixproof.netlify.app/ is a read-only, sanitized view of saved evidence. It does not invoke a model, scan code, execute attacks, approve candidates, or deploy repaired software.

4. Experimental Method

4.1 Controlled fixtures

The benchmark contains one fixture for reflected XSS, one for SQL injection, and one for path traversal. Each fixture defines a vulnerable operation, synthetic data or files, a baseline attack, and benign-behavior expectations. The fixtures make the expected behavior reproducible within the semester but sacrifice the diversity and complexity of production applications.

4.2 Primary-v1

Primary-v1 scheduled five initial repair calls per fixture for 15 calls total. Within each fixture, the generator (gpt-5.2), prompt template (1.0), and model tool access (none) were held constant. The repair model received the target finding and focused source context, but it did not receive the hidden runtime test implementation. Each saved candidate received syntax, candidate SAST, targeted HTTP or browser security, and functional checks.

The deterministic policy could produce REJECT, READY_FOR_HUMAN_REVIEW, or NEEDS_HUMAN_ADJUDICATION. Readiness was not an approval. Ten SAST/runtime disagreements required original human conflict reviews, and those records were preserved separately from automated decisions.

4.3 Supplemental-v1

Supplemental-v1 was registered before executing its candidate runs. It reused the same 15 saved candidates, made no new model calls, and did not rerun the primary SAST evaluation. The protocol added security, behavioral-parity, and robustness contracts for missing, repeated, encoded, punctuation, Unicode, boundary, and related input conditions. The supplemental layer contains 140 candidate observations. Later human decisions qualify the evidence but do not rewrite primary-v1.

4.4 PATH-S06 extension-v1

Windows denied symbolic-link creation for the five supplemental traversal runs, so PATH-S06 was recorded as inconclusive rather than pass or fail. A separate protocol was hash-locked and executed on WSL2 Ubuntu on September 30. It reused the vulnerable baseline and the same five saved traversal candidates. The baseline had to disclose a controlled outside-file marker through the link to show that the test topology worked. Candidate success required no marker disclosure and an HTTP status of 400, 403, or 404.

This extension did not replace the five original inconclusive observations and did not enter supplemental-v1's 140-observation denominator.

5. Results

5.1 Primary candidate outcomes

Primary case	Candidate instances	Target SAST resolved	Complete security suite passed	Complete functional suite passed	Automated route
Reflected XSS	5	0/5	5/5	5/5	Five need adjudication
SQL injection	5	5/5	5/5	5/5	Five ready for human review
Path traversal	5	0/5	5/5	5/5	Five need adjudication
Total	15	5/15	15/15	15/15	Five ready; ten need adjudication

The security and functional columns count candidates whose full registered primary suite passed; they are not counts of individual test observations. Primary-v1 contains 40 security observations and 60 functional observations, all passing. Ten of the 15 candidates therefore demonstrate disagreement between a persistent target scanner finding and passing registered runtime checks. Five SQLi candidates removed the target finding and passed the fixed checks. No primary candidate demonstrated the opposite failure mode of removing the target finding while remaining exploitable under the registered primary security suite.

5.2 Supplemental and extension observations

Evidence layer	Category	Observations	Pass	Fail	Inconclusive
Primary-v1	Security	40	40	0	0
Primary-v1	Functional	60	60	0	0
Supplemental-v1	Security	60	55	0	5
Supplemental-v1	Behavioral parity	45	40	5	0
Supplemental-v1	Robustness	35	25	10	0
PATH-S06 extension-v1	Security	5	0	5	0

These layers must not be pooled into a single pass rate. Their protocols, categories, dates, platforms, and decision rules differ. The browser evidence viewer exposes 245 case rows: 100 primary observations, 140 supplemental observations, and five extension observations, each retaining its evidence layer.

Four XSS candidates failed XSS-P01 because their handling of a missing name changed the frozen output from Hello undefined to Hello . XSS 03 passed all nine supplemental XSS cases. The four parity failures do not show continued XSS exploitability; they show that an otherwise passing security repair changed the registered benign behavior.

All five SQLi candidates passed three supplemental security and two parity cases, but each failed the separate SQL-R01 repeated-parameter contract by returning HTTP 200 with an empty result instead of the registered HTTP 400. This is a bounded robustness limitation rather than a retroactive primary injection failure.

Traversal attempt 01 rejected the valid in-root filename ..notes.txt and failed a repeated-parameter rule. Other traversal candidates varied on malformed-input robustness. All five executable supplemental traversal security cases passed for every candidate, but the symbolic-link case was inconclusive on Windows.

The WSL2 extension resolved that environment limitation for one registered topology. The vulnerable baseline disclosed the marker, confirming the positive control. Every saved traversal candidate then returned HTTP 200 and disclosed the same controlled outside-file marker. Candidate totals were zero pass, five fail, and zero inconclusive. This is evidence of an exploitable symbolic-link escape in the registered extension, not a claim about every path-traversal technique or platform.

5.3 Human decisions

Human evidence layer	Records	Recorded interpretation
Original primary conflict reviews	10	Seven acceptances and three requests for more testing under the original evidence
Supplemental-v1 later decisions	14	Five rejections, five bounded SQLi acceptances, and four requests for more testing
PATH-S06 extension qualifications	5	Five later traversal rejections; earlier records preserved
Total displayed records	29	Separate authorities and dates; not one approval rate

The later records intentionally do not overwrite earlier decisions. For example, Traversal 02-05 first requested more evidence because the Windows symbolic-link test was inconclusive. The later WSL2 results support separate rejections. This preserves the history of what was known when each decision was recorded.

6. Interpretation

The results support three distinct conclusions. First, the SQLi candidates provide bounded evidence of repair: they use parameterized queries, remove the target finding, pass the registered attack and parity checks, and return no unauthorized rows. Their repeated-parameter behavior remains a disclosed robustness limitation.

Second, passing security checks can coexist with a behavioral regression. Four XSS candidates blocked the registered payloads but changed the missing-input contract. This is why FixProof stores security and functional evidence separately instead of reporting a single success flag.

Third, a passing fixed suite can leave a security gap. The traversal candidates passed the executable primary and supplemental direct-input cases, yet the later PATH-S06 extension demonstrated marker disclosure for all five. The correct conclusion is not that fixed tests are useless; it is that a pass means the candidate met the registered oracle, not that all attack classes were excluded.

7. Reproducibility and Traceability

A clean-package rehearsal exported a disposable Git snapshot, created a new Python environment, restored pinned Python dependencies and all six committed Node dependency trees, installed the Playwright browser, verified saved evidence, ran 131 repository tests, and executed the controlled demonstration suite. The readiness and guided-demo runs both passed. The verifier confirmed 15/15 primary attempts, 10/10 original conflict reviews, control/evidence separation, and artifact bindings.

The 131 tests validate repository machinery such as parsers, runners, policies, reports, integrity checks, and controlled demonstrations. They are not 131 new candidate attacks and do not rerun every historical application trial. The clean rehearsal increases confidence that the package can reproduce its checks from the documented snapshot; it is not independent replication by a separate research team.

The public evidence package presents 15 candidate summaries, 245 separately labeled case rows, and 29 human records. A dated live check on October 22 found that selected deployed artifacts matched the local release files by SHA-256.

8. Limitations and Threats to Validity

• The study contains one controlled fixture per CWE. The five repair calls per fixture are repeated generations against the same application, not 15 independent applications.

- All five saved SQLi candidates contained identical source, so repeated calls do not create five distinct implementation designs for that case.
- The registered payloads and benign-behavior contracts are finite. Passing them is bounded evidence, not proof of application-wide security.
- The primary study observed no candidate that removed the target SAST finding while remaining exploitable under its primary registered security suite. FixProof therefore cannot estimate a detection rate for that failure mode.
- Static analysis and runtime tests answer different questions. A persistent finding is not proof of exploitability, and disappearance is not proof of correctness.
- PATH-S06 required a separately frozen WSL2 execution because Windows denied symlink creation. The result is conclusive for the registered Linux topology, not for every operating system or file-system configuration.
- Copied workspaces are not isolation for arbitrary hostile repositories.
- Clean installation reported dependency advisories in the frozen fixture lockfiles. Running an automatic dependency fix would alter the evaluated snapshot, so the advisories are disclosed as a maintenance limitation and remain separate from candidate-test outcomes.
- The researcher developed the system and performed the human reviews. A second reviewer could strengthen the adjudication evidence and expose ambiguous decision criteria.

9. Draft Conclusion and Future Work

FixProof demonstrates a validation-oriented workflow in which AI-generated repairs are proposals rather than approvals. In three controlled Express fixtures, static and runtime evidence disagreed for ten primary candidates; supplemental tests identified parity and robustness gaps that the primary suite did not represent; and a separately registered symbolic-link extension exposed a concrete traversal failure in five saved candidates. Preserving each layer and its human interpretation made those changes in evidence traceable.

The defensible contribution is the evidence and authority structure, not a claim of model or system superiority. Future work should apply the workflow to additional repositories and independently designed validators, add a second reviewer, study broader attack surfaces, and evaluate how repair decisions change when test coverage changes. Those extensions should be versioned rather than mixed into the frozen primary and supplemental records.

10. Draft Completeness and Remaining Work

The current draft includes the sections needed to demonstrate the project's direction and evidence. Before final submission, I will:

- add a concise architecture figure and one candidate-level comparison figure;
- tighten the prose to the course page limit and apply the required template;

- verify every citation, URL, table denominator, and candidate-specific claim;
- finish the final source/evidence manifest and SHA-256 list;
- check DOCX and PDF rendering, accessibility, and public-link permissions; and
- incorporate useful course feedback without modifying frozen evidence.

11. Generative AI Disclosure

OpenAI ChatGPT/Codex assisted with project planning, implementation, debugging, test orchestration, documentation, report organization, and editing. FixProof separately used the OpenAI API to generate remediation candidates, with runtime prompts and responses retained in the experiment artifacts. For human review, I inspected the candidate patches, payloads and inputs, security results, functional or parity results, and recorded outputs. I used AI assistance to help identify issues that required checking, but I verified the cited evidence, made the recorded decisions, and remain responsible for the project and this draft.

References

Kulsum, U., Zhu, H., Xu, B., & d'Amorim, M. (2024). A case study of LLM for automated vulnerability repair: Assessing impact of reasoning and patch validation feedback. *Proceedings of the 1st ACM International Conference on AI-Powered Software*. https://doi.org/10.1145/3664646.3664770

OWASP Foundation. (n.d.). *Web Security Testing Guide, version 4.2*. https://wstg.owasp.org/v4.2/

Pearce, H., Ahmad, B., Tan, B., Dolan-Gavitt, B., & Karri, R. (2022). Asleep at the keyboard? Assessing the security of GitHub Copilot's code contributions. *2022 IEEE Symposium on Security and Privacy*. https://doi.org/10.1109/SP46214.2022.9833571

Pearce, H., Tan, B., Ahmad, B., Karri, R., & Dolan-Gavitt, B. (2023). Examining zero-shot vulnerability repair with large language models. *2023 IEEE Symposium on Security and Privacy*. https://doi.org/10.1109/SP46215.2023.10179324

Semgrep. (n.d.). *Run rules*. https://semgrep.dev/docs/running-rules

Supporting Evidence Index

- Primary protocol and results: docs/study-protocol-v1.md and docs/primary-results.md
- Supplemental protocol and results: docs/supplemental-protocol-v1.md and docs/supplemental-results-v1.md
- PATH-S06 protocol and results: docs/path-s06-extension-v1-protocol.md and docs/path-s06-extension-v1-results.md
- Reproducibility guidance: docs/reproducibility.md
- Clean-package validation: docs/coursework/review-2026-10-22/Clean-package-reproduction-validation.md
- Public implementation repository: https://github.com/ttran97/fixproof
- Read-only evidence viewer: https://fixproof.netlify.app/